

ViniGames — Estructura del Sistema de Archivos y Arquitectura del Código

Proyecto: ViniGames

Framework: Next.js 16 (App Router) + React 19 + TypeScript + Tailwind CSS v4 + Supabase

Estándar de Organización: Arquitectura modular orientada a funcionalidades (Feature-Driven & App Router Structure).

1. Árbol de Directorios Completo del Proyecto

```
ViniGames/
├── docs/                                # Documentación técnica, académica y guías
│   ├── INFORME_GENERAL_PROYECTO.md      # Informe estructurado general
│   ├── GUIA_TECNOLOGIAS_Y_FLUJOS.md     # Guía didáctica de estudio
│   ├── ESTRUCTURA_SISTEMA_ARCHIVOS.md  # Este documento (mapa del código)
│   ├── PLAN_IMPLEMENTACION_EQUIPO.md    # Plan de trabajo por integrante
│   └── GUIA_GIT_GITHUB_EQUIPO.md        # Guía de Git y flujo de ramas
├── Informe/                             # Archivos fuente del informe universitario
│   └── Vinigames SRS (2).pdf             # Documento PDF base de la materia
├── VINIGAMES_KNOWLEDGE_BASE.md          # Base de conocimiento maestra (Single Source of Truth)
├── vinigames/                           # CÓDIGO FUENTE DE LA APLICACIÓN WEB
│   ├── .env.local.example               # Plantilla de variables de entorno
│   ├── .gitignore                       # Reglas de exclusión para Git
│   ├── eslint.config.mjs                # Configuración de ESLint 9
│   ├── next.config.ts                   # Configuración de Next.js 16 (imágenes remotas)
│   ├── package.json                     # Dependencias y scripts de ejecución
│   ├── postcss.config.mjs               # Configuración de PostCSS para Tailwind v4
│   ├── tsconfig.json                    # Configuración estricta de TypeScript
│   └── middleware.ts                    # Route Guard para protección de rutas y RBAC
│   ├── public/                          # Recursos estáticos públicos
│   │   ├── avatars/                     # Iconos de avatares predeterminados
│   │   │   ├── ninja.png
│   │   │   ├── cyber.png
│   │   │   ├── mech.png
│   │   │   └── wizard.png
│   │   ├── games/                       # Portadas y banners de prueba
│   │   ├── icons/                       # SVGs e iconos estáticos
│   │   └── favicon.ico
│   └── app/                              # Rutas y páginas (Next.js 16 App Router)
│       ├── layout.tsx                    # Root Layout (Fuentes, Providers globales)
│       ├── globals.css                   # Variables CSS y estilos base de Tailwind v4
│       ├── loading.tsx                   # UI de carga global (Skeleton loader)
│       ├── not-found.tsx                 # Página 404 personalizada Gamer
│       └── error.tsx                     # Manejador global de errores en cliente
│       ├── (auth)/                       # Grupo de rutas de autenticación (sin layout comercio)
│       │   ├── login/
│       │   │   └── page.tsx               # Pantalla de Inicio de Sesión
│       │   └── onboarding/
│       │       ├── layout.tsx             # Barra de progreso del Onboarding (Pasos 1 a 4)
│       │       ├── step-1/page.tsx        # Paso 1: Identidad Gamer y Avatar
│       │       ├── step-2/page.tsx        # Paso 2: Selección de Géneros Favoritos
│       │       ├── step-3/page.tsx        # Paso 3: Ponderación Gamer DNA
│       │       ├── step-4/page.tsx        # Paso 4: Credenciales y Contraseña
│       │       └── welcome/page.tsx        # ¡Misión Completada! Bienvenida y Nivel 1
│       ├── (store)/                      # Grupo de rutas de la tienda (comparten Header Game
│       │   └── layout.tsx                 # Header persistente (XP, Racha, GameCoins) y Footer
```

```

├── page.tsx # Home (Hero Banner, Ofertas, Recomendados DNA)
├── catalog/
│   └── page.tsx # Catálogo completo con filtros reactivos
├── games/
│   └── [slug]/
│       └── page.tsx # Ficha técnica del juego, Reseñas y Resumen IA
├── cart/
│   └── page.tsx # Carrito de compras y Checkout simulado
├── wishlist/
│   └── page.tsx # Lista de deseos con alertas de rebaja
├── library/
│   └── page.tsx # Biblioteca privada (Horas, Jugar, Calificar)
├── gamification/
│   └── page.tsx # Hub de Rachas de 7 días, Niveles y Medallas
├── chat/
│   └── page.tsx # ViniChat Asistente Virtual IA
├── profile/
│   └── page.tsx # Perfil público/privado con gráfico Gamer DNA
├── admin/ # Panel Administrativo ViniAdmin (Protegido por Rol)
│   ├── layout.tsx # Sidebar administrativo y verificación RBAC
│   ├── page.tsx # Dashboard principal (KPIs de ventas, gráficos)
│   ├── games/
│   │   ├── page.tsx # Tabla CRUD del catálogo de juegos
│   │   └── new/page.tsx # Formulario para publicar nuevo videojuego
│   ├── sales/
│   │   └── page.tsx # Registro transaccional y exportación CSV
│   ├── reviews/
│   │   └── page.tsx # Cola de moderación comunitaria
│   └── users/
│       └── page.tsx # Gestión de usuarios y asignación de roles
├── actions/ # Server Actions (Mutaciones de datos en servidor)
│   ├── auth.actions.ts # Registro, Login, Logout, Guardado Onboarding
│   ├── cart.actions.ts # Agregar al carrito, Checkout simulado
│   ├── games.actions.ts # CRUD de juegos (Admin) y filtros
│   ├── library.actions.ts # Registro de horas jugadas y estados
│   ├── reviews.actions.ts # Crear reseña, Votar utilidad (+/-)
│   ├── gamification.actions.ts # Cálculo diario de racha y reclamo de XP
│   └── chat.actions.ts # Envío de mensajes al webhook de n8n / DeepSeek
├── api/ # Endpoints REST / Webhooks
│   ├── webhooks/
│   │   └── n8n/route.ts # Endpoint para recibir eventos de n8n
│   └── health/route.ts # Health-check del sistema
├── components/ # Componentes de React reutilizables
│   ├── ui/ # Primitives de interfaz (Átomos / Botones / Modales)
│   │   ├── button.tsx
│   │   ├── input.tsx
│   │   ├── badge.tsx
│   │   ├── card.tsx
│   │   ├── modal.tsx
│   │   ├── progress.tsx
│   │   └── toast.tsx

```

└─ layout/	# Componentes de estructura
└─ header.tsx	# Barra de navegación con widgets de gamificación
└─ footer.tsx	# Pie de página institucional
└─ admin-sidebar.tsx	# Menú lateral del panel administrativo
└─ store/	# Componentes de la tienda
└─ game-card.tsx	# Tarjeta de juego con precio y descuento
└─ hero-banner.tsx	# Banner principal rotativo
└─ filter-sidebar.tsx	# Filtros por categorías múltiples y precio
└─ cart-drawer.tsx	# Desplegable lateral del carrito
└─ review-feed.tsx	# Listado de reseñas con votos de utilidad
└─ gamification/	# Componentes del sistema de juego
└─ streak-card.tsx	# Widget del calendario de racha de 7 días
└─ xp-bar.tsx	# Barra animada de experiencia hacia el nivel N
└─ gamer-dna-radar.tsx	# Gráfico visual de los 4 rasgos del jugador
└─ badge-grid.tsx	# Galería de medallas bloqueadas y desbloqueadas
└─ chat/	# Componentes de ViniChat
└─ chat-sidebar.tsx	# Lista de temas de conversación
└─ chat-window.tsx	# Contenedor de mensajes con scroll automático
└─ message-bubble.tsx	# Burbuja de mensaje con soporte Markdown
└─ product-chat-card.tsx	# Mini-tarjeta de compra embebida en la IA
└─ admin/	# Componentes de ViniAdmin
└─ stats-kpi-card.tsx	# Tarjetas de ingresos y métricas
└─ revenue-chart.tsx	# Gráfica de transacciones en el tiempo
└─ data-table.tsx	# Tabla reutilizable con ordenamiento y paginación
└─ game-form-modal.tsx	# Modal con formulario de subida de juegos
└─ lib/	# Utilidades, clientes externos y configuraciones
└─ supabase/	
└─ client.ts	# Cliente Supabase para el navegador (Client Components)
└─ server.ts	# Cliente Supabase para Server Components & Actions
└─ middleware.ts	# Lógica de sincronización de sesión en Middleware
└─ schemas/	# Esquemas de validación Zod (Single Source of Validation)
└─ auth.schema.ts	# Esquemas de Login y Onboarding en 4 pasos
└─ game.schema.ts	# Esquema de validación para creación de juegos
└─ review.schema.ts	# Validación de puntuación y contenido de reseñas
└─ order.schema.ts	# Validación de checkout y métodos de pago
└─ gamification/	# Lógica matemática de gamificación
└─ level-calculator.ts	# Función: calcular nivel a partir de total_xp
└─ streak-manager.ts	# Función: verificar y calcular racha diaria
└─ n8n/	
└─ webhook-client.ts	# Cliente tipado para disparar webhooks a n8n
└─ utils.ts	# Funciones auxiliares (formateo de moneda Bs., fecha)
└─ types/	# Definiciones de tipos TypeScript (.d.ts / interfaces)
└─ database.types.ts	# Tipos autogenerados de Supabase PostgreSQL
└─ game.types.ts	# Interfaces de Videojuego, Categoría, Media
└─ user.types.ts	# Interfaces de Perfil Gamer, Gamer DNA

```
| gamification.types.ts  
| order.types.ts
```

```
# Interfaces de Medallas, Rachas, Logros  
# Interfaces de Órdenes, Items, Recibos
```

2. Convenciones de Nomenclatura y Estilo de Código

1. **Archivos de Componentes:** Formato `kebab-case.tsx` (ej: `game-card.tsx`, `hero-banner.tsx`, `streak-card.tsx`).
2. **Archivos de Server Actions:** Sufijo `.actions.ts` (ej: `cart.actions.ts`, `auth.actions.ts`).
3. **Archivos de Validación Zod:** Sufijo `.schema.ts` (ej: `auth.schema.ts`, `review.schema.ts`).
4. **Archivos de Tipos TypeScript:** Sufijo `.types.ts` (ej: `game.types.ts`, `user.types.ts`).
5. **Componentes React:** PascalCase en la función exportada (ej: `export function GameCard(...)`).
6. **Rutas Dinámicas:** Encapsuladas entre corchetes (ej: `app/(store)/games/[slug]/page.tsx`).
7. **Rutas Agrupadas:** Encapsuladas entre paréntesis sin alterar la URL pública (ej: `(auth)`, `(store)`).